# Multi-Stage Spotify Recommendation System

Applying Deep Learning to Sequential Music Retrieval

Youssef Miled, Idris Houir Alami, Emeryc Chuttarsing

**Abstract**

## 1 Introduction

### 1.1 Presentation link

### 1.2 Problem Formulation

Considering that there are $N$ songs in a playlist, determine the $M$ songs that will follow them based on a library of 2.26 tracks. This is a sequential collaborative filtering problem. The challenge involves achieving relevance by recommending the next likely song in the playlist while avoiding redundancy.

### 1.3 Dataset and Pipeline

**Dataset:** Spotify Million Playlist Dataset: 1,000,000 playlists, 2,262,292 unique tracks, ≈45M interactions. Split: 70% train (700K), 15% val (150K), 15% test (150K).

**Pipeline:** Playlist Context → Stage 1 (ALS, top 1K) → Stage 2 (Ranking, 1K→100) → Stage 3 (Re-ranking with MMR diversity, 100→20–30) → Web UI.

## 2 Methodology and results

### 2.1 Stage 1: Candidate generation

#### 2.1.1 Approach

Stage 1 solves the high-recall retrieval subproblem: given an observed playlist prefix $S = \{i_1, \ldots, i_t\}$, retrieve 1000 plausible next tracks from the full catalog of 2.26M tracks. The input features are implicit interactions between playlists and tracks; the label for evaluation is the held-out final track. Because the interaction matrix is extremely sparse, we used implicit-feedback ALS as a practical first-stage collaborative filter. ALS factorizes a playlist × track matrix $\mathbf{R}$ into playlist and track embeddings, $\mathbf{R} \approx \mathbf{U}\mathbf{V}^T$, with $\mathbf{V} \in \mathbb{R}^{2.26M \times 128}$. It minimizes a confidence-weighted squared loss with L2 regularization:

$$\min_{\mathbf{U},\mathbf{V}} \sum_{p,i} c_{pi}(r_{pi} - \mathbf{u}_p^\top \mathbf{v}_i)^2 + \lambda(\|\mathbf{U}\|_2^2 + \|\mathbf{V}\|_2^2), \quad c_{pi} = 1 + \alpha r_{pi}.$$

We used binary implicit feedback, factors=128, iterations=15, $\alpha = 40$, and $\lambda = 0.01$. At inference time, a new playlist has no learned row in $\mathbf{U}$, so we form a query embedding by mean-pooling its observed tracks, $\mathbf{u}_q = |S|^{-1} \sum_{i \in S} \mathbf{v}_i$, score all tracks by $\mathbf{u}_q^\top \mathbf{v}_j$, mask tracks already in the playlist, and return the top 1000. This stage is intentionally order-agnostic: its role is broad filtering, while Stage 2 handles sequence-aware ranking.

Before choosing ALS, we tested two neural alternatives. A two-tower model used a shared 128-dimensional embedding table, mean pooling, an MLP projection, L2 normalization, and in-batch InfoNCE with AdamW. Track2Vec trained skip-gram embeddings over playlist sequences. Both preserved the same mean-pooled retrieval bottleneck at inference, which made them poor matches for the first-stage retrieval constraint under our compute budget.

#### 2.1.2 Results

| Method | R@100 | R@1000 | Time |
|---|---|---|---|
| Two-Tower (neural) | 0.029 | 0.058 | 10h |
| Track2Vec (Skip-gram) | 0.128 | 0.403 | 6h |
| **ALS (chosen)** | **0.151** | **0.407** | **8m** |

Table 1: ALS outperforms neural baselines 5–7× while training 75× faster. Mean-pooling limits R@1000 to ≈ 40%.

**Analysis:** ALS was the best Stage 1 tradeoff: it reached R@100=0.151 and R@1000=0.407 on all 150K test playlists in roughly 8 minutes, while the two-tower model trained for about 10 hours and remained close to the random InfoNCE loss ($\log(512) \approx 6.24$). Track2Vec learned useful local co-occurrence structure during training, but mean-pooling erased much of that sequence information at retrieval time, leaving R@1000 nearly identical to ALS and R@100 lower. The main error mode is therefore high bias rather than high variance: validation and test behavior were similar, but the query representation cannot distinguish "song A then song B" from "song B then song A." This motivates the full pipeline design. Stage 1 supplies a fast, high-coverage candidate set; Stage 2 then applies causal self-attention to recover the ordering signal that ALS deliberately ignores.

### 2.2 Stage 2: Sequential Re-ranking (SASRec)

#### 2.2.1 Approach

A causal Transformer encoder reads the playlist as an ordered sequence and emits a query vector at the final non-pad position, scored against item embeddings: $\text{score}_j = \mathbf{q} \cdot \mathbf{e}_j$. Architecture: 2 layers, 2 heads, $d_{\text{model}} = 128$, $d_{\text{ffn}} = 256$, max sequence length 50, pre-LayerNorm with dropout 0.2. The item embedding table is initialized from Stage 1's $\mathbf{V}$ to inherit the co-occurrence structure already learned by ALS; positional embeddings are learned from scratch; ID 0 is reserved as a PAD token.

Training uses causal language modeling: at every position $i$ of every training playlist, predict song $i + 1$. Loss is softmax cross-entropy over a per-position score vector composed of {positive, 128 hard negatives sampled from Stage 1's top-1000 candidates for the playlist, 128 random negatives uniform over the vocabulary}. The hard-negative term aligns training with the inference distribution (Stage 2 re-ranks within Stage 1's candidate set); the random-negative term prevents the model from over-specializing to the candidate-set shape.

Optimizer: AdamW with two parameter groups (discriminative fine-tuning) — transformer layers at $\eta = 10^{-3}$, embedding table at $\eta = 10^{-4}$ to preserve the ALS initialization. Weight decay 0.01, linear warmup over 1000 steps then cosine decay, batch size 256, bf16 autocast, gradient clipping at max-norm 1.0. Trained on a Modal A10G GPU; 15-epoch budget with early stopping on validation NDCG@10 (patience: 3 evaluation checkpoints). Wall-clock ∼67 minutes; cost ∼$3 on Modal credits.

Evaluation: held-out last song. Two scoring modes — *pipeline* (score against Stage 1's top-1000 candidates for that playlist, the operational setting) and *unconstrained* (score against the full 2.26M-track vocabulary, a model-quality probe independent of the candidate filter).

#### 2.2.2 Results

| Method | R@10 | R@100 | R@1000 | NDCG@10 |
|---|---|---|---|---|
| Stage 1 (ALS, baseline) | 0.035 | 0.151 | 0.407 | 0.018 |
| Stage 2 unconstrained | 0.022 | 0.070 | 0.188 | 0.013 |
| **Stage 2 pipeline** | **0.082** | **0.236** | 0.407† | **0.046** |

Table 2: Stage 2 evaluation on the 150K held-out test playlists. † Pipeline R@1000 is locked at Stage 1's value because Stage 2 re-ranks within Stage 1's 1000 candidates and cannot change recall at that depth.

**Analysis:** The pipeline configuration improves over Stage 1 alone by +56% on R@100 (0.151 → 0.236) and +156% on NDCG@10 (0.018 → 0.046), confirming that sequence-aware re-ranking captures signal that Stage 1's mean-pool inference discards. More striking is the gap between pipeline and unconstrained modes: pipeline R@100 (0.236) is $\sim 3.4\times$ unconstrained R@100 (0.070). Stage 2 *alone*, without Stage 1's candidate filter, is a *worse* retriever than Stage 1 itself — because the fine-tuned embeddings have specialized for ranking within a candidate set, not for general retrieval over the full vocabulary. This empirically justifies the two-stage architecture: Stage 1 handles coarse filtering, Stage 2 handles fine ranking, and neither is sufficient on its own. Training plateaued at epoch 6 (Table 6 in the appendix); the 2-layer architecture saturates quickly on this dataset, and the aspirational targets of R@100 > 0.30 and NDCG@10 > 0.15 were not reached, likely requiring a larger model and a longer training budget.

### 2.3 Stage 3: Diversity re-ranking (MMR)

#### 2.3.1 Approach

Greedy Maximal Marginal Relevance over Stage 2's top-100 per playlist, producing a final top-$K = 20$. At each step, given the already-selected set $S$:

$$\text{MMR}(c) = \lambda \cdot \text{rel}(c) - (1 - \lambda) \cdot \max_{s \in S} \cos(\mathbf{e}_c, \mathbf{e}_s)$$

where $\text{rel}(c)$ is Stage 2's relevance score for candidate $c$ (min-max normalized within the playlist's 100 candidates) and $\mathbf{e}_c$ is the L2-normalized fine-tuned item embedding from Stage 2. The first pick uses pure relevance ($S = \emptyset$).

The original project specification defined the diversity term in audio-feature space (tempo, energy, valence, danceability) with additional rule-based filters (artist cap, tempo continuity, energy smoothing). **None of this data was accessible within the project's scope**: Spotify Web API access for 2.26M tracks was infeasible, and no pre-fetched audio-feature dump was available. We therefore replaced audio-feature similarity with cosine similarity in Stage 2's learned embedding space, which encodes co-occurrence and sequential substitutability between songs — exactly the structural property MMR's diversity term requires. The audio-feature-based post-filters were dropped entirely.

Implementation is pure NumPy, fully vectorized over batches of 1024 playlists. Total runtime: $\sim 7$ seconds per $\lambda$ value on all 150K test playlists. Cost: \$0 (no Modal compute, no GPU).

#### 2.3.2 Results

We swept $\lambda \in \{0.3, 0.5, 0.7\}$ and compared against the Stage 2 raw top-20 baseline (take Stage 2's top-20 without any re-ranking). Diversity is measured by *intra-list diversity* (ILD), the mean

pairwise cosine *distance* $(1 - \cos)$ among the final 20 songs in each playlist, averaged across all 150K test playlists.

| **Configuration** | **Recall@20** | **ILD** |
|---|---|---|
| Stage 2 raw top-20 (baseline) | 0.119 | 0.368 |
| MMR $\lambda = 0.3$ (diversity-leaning) | 0.098 | 0.496 |
| **MMR $\lambda = 0.5$ (reported)** | **0.114** | **0.433** |
| MMR $\lambda = 0.7$ (relevance-leaning) | 0.117 | 0.393 |

Table 3: Stage 3 evaluation on 150K test playlists, $K = 20$. Higher ILD indicates more variety in the final selection; bounded in $[0, 2]$.

**Analysis:** The $\lambda$ knob produces a clean monotonic tradeoff between recall and diversity, as the algorithm predicts. At the reported configuration ($\lambda = 0.5$), MMR sacrifices $\sim 4\%$ recall ($0.119 \rightarrow 0.114$) for an $\sim 18\%$ gain in intra-list diversity ($0.368 \rightarrow 0.433$). The $\lambda = 0.7$ point is particularly informative: diversity improves by $+6.6\%$ at only $-1.1\%$ recall, suggesting that Stage 2's raw top-20 is unnecessarily redundant. The fact that ILD changes meaningfully under MMR re-ordering also confirms that Stage 2's fine-tuned embeddings carry useful structural information *beyond* the relevance score they were trained to produce: songs that score similarly in relevance are not necessarily close in embedding space, so MMR's diversity term has a real signal to exploit.

### 2.4 Web interface

#### 2.4.1 Stack

## 3 Safety, Security, and Ethics

In a large-scale deployment scenario, our recommendation system could amplify the popularity bias because it is built based on the previous listening history of the users. Thus, it will recommend popular artists more often compared to emerging or minority ones, which can negatively affect the overall diversity of discovered music. Besides, the sequential recommendation algorithms aim to maximize engagement with music, which might foster addictive behavior or create filter bubbles by narrowing down content diversity. Moreover, the main privacy issue arises from the extensive use of user interaction history in large-scale recommendation systems. Interaction history may contain sensitive information such as personal preference, mood, culture, or even ethnicity. Although interaction data is anonymized, embedding models might memorize and potentially expose rare patterns in the listening history if not used properly. From the security point of view, recommendation systems might become targets of manipulation attacks, such as creating fake playlists or bots for artificial interactions to boost certain songs' performance in recommendations. Lastly, our model suffers from the cold-start problem for unpopular artists, which causes discrimination toward less represented content. However, since our approach is based on simple collaborative filtering and transformer ranking algorithms under resource-limited constraints, there would still need to be further precautions to make this project applicable for production use. In the event of production use, we should additional considerations for fairness-aware reranking, user privacy concerns, and the transparency of recommendation decisions.

# 4 Conclusion

# References

[1] Bennet et al. (2020). "Collaborative Filtering via Alternating Least Squares," *Implicit library documentation.*

[2] Spotify. "Million Playlist Dataset Challenge," RecSys Challenge 2018.

[3] Wang-Cheng et al. (2018). "Self-Attentive Sequential Recommendation," ICCM.

# A Detailed Results Tables

## A.1 Two-Tower Neural Network

| Epoch | Train Loss | Val Loss | Time/Epoch | Cumulative |
|---|---|---|---|---|
| 1 | 6.28 | 6.24 | 1h 25m | 1h 25m |
| 2 | 6.23 | 6.20 | 2h 18m | 3h 43m |
| 3 | 6.17 | 6.12 | 2h 28m | 6h 11m |
| 4 | 6.11 | — | 2h 20m | 8h 31m |

Table 4: Two-Tower training. Random baseline = $\log(512) \approx 6.24$. Model barely improved.

## A.2 ALS Full Results

Table 5: ALS matrix factorization configuration and evaluation results.

| Property | Value |
|---|---|
| Dataset | Spotify Million Playlist Dataset |
| Train / Val / Test split | 700K / 150K / 150K playlists |
| Unique tracks | 2,262,292 |
| Embedding dimension | 128 |
| ALS iterations | 15 |
| Regularization | 0.01 |
| Confidence scaling $\alpha$ | 40 |
| Training time | $\sim$8 minutes |
| Recall@10 | 0.0352 |
| Recall@50 | 0.1023 |
| Recall@100 | 0.1513 |
| Recall@500 | 0.3188 |
| Recall@1000 | 0.4067 |
| NDCG@10 | 0.0179 |

# B Stage 2 Training Trajectory

The full SASRec run early-stopped at epoch 14 of a 15-epoch budget. Validation NDCG@10 peaked at epoch 6 (best checkpoint) and plateaued thereafter; the early-stopping rule (no improvement for 3 consecutive validation checkpoints) triggered at epoch 14.

| Epoch | Train Loss | Val R@10 | Val R@100 | Val NDCG@10 | Wall (min) |
|---|---|---|---|---|---|
| 2 | 3.363 | 0.081 | 0.236 | 0.045 | 9.3 |
| 4 | 3.101 | 0.082 | 0.239 | 0.046 | 18.7 |
| **6** | **2.976** | **0.085** | **0.239** | **0.049** | 28.3 |
| 8 | 2.893 | 0.086 | 0.242 | 0.049 | 37.9 |
| 10 | 2.837 | 0.086 | 0.240 | 0.049 | 47.5 |
| 12 | 2.807 | 0.085 | 0.240 | 0.048 | 57.1 |
| 14 | 2.794 | 0.085 | 0.241 | 0.048 | 66.8 |

Table 6: SASRec training. Validation evaluated every 2 epochs on the first 10K val playlists in pipeline mode. Train loss kept decreasing but validation metrics plateaued by epoch 6 (bold = best checkpoint). Early-stopped at epoch 14.

# C Stage 2 Configuration

Key hyperparameters from `stage_2/config.yaml`:

```
model:
  d_model: 128
  n_layers: 2
  n_heads: 2
  ffn_dim: 256
  max_seq_len: 50
  dropout: 0.2
  vocab_size: 2262293 # 2262292 ALS tracks + 1 PAD

training:
  batch_size: 256
  epochs: 15 # early-stopped at 14
  warmup_steps: 1000
  peak_lr_transformer: 1.0e-3
  peak_lr_embedding: 1.0e-4
  weight_decay: 0.01
  grad_clip: 1.0
  precision: bf16
  num_hard_negs: 128
  num_random_negs: 128
  val_every_n_epochs: 2
  early_stopping_patience_evals: 3
```

# D Stage 3 Detailed Metrics

The full $\lambda$ sweep alongside the Stage 2 raw top-20 baseline. Each row is averaged over all 150K held-out test playlists.

| Config | Recall@20 | ILD | $\Delta$ Recall | $\Delta$ ILD |
|---|---|---|---|---|
| Stage 2 raw top-20 | 0.1186 | 0.3682 | — | — |
| MMR $\lambda = 0.3$ | 0.0975 | 0.4958 | $-17.8\%$ | $+34.7\%$ |
| **MMR** $\lambda = 0.5$ | **0.1135** | **0.4327** | $-4.3\%$ | $+17.5\%$ |
| MMR $\lambda = 0.7$ | 0.1173 | 0.3925 | $-1.1\%$ | $+6.6\%$ |

Table 7: Stage 3 $\lambda$ sweep on the full 150K test set, $K = 20$. $\Delta$ columns are relative to the Stage 2 raw top-20 baseline.

# E Stage 1 Inference Code

```python
import numpy as np, json

item_factors = np.load('checkpoints/als_item_factors.npy')
uri_to_id = json.load(open('checkpoints/uri_to_id.json'))

def get_candidates(playlist_uris, k=1000):
    ids = [uri_to_id[u] for u in playlist_uris
               if u in uri_to_id]
    user_emb = item_factors[ids].mean(axis=0)
    scores = user_emb @ item_factors.T
    scores[ids] = -1e9
    top_k = np.argpartition(scores, -k)[-k:]
    return top_k[np.argsort(scores[top_k])[::-1]]
```

# F Configuration

Located at `stage_1/config/config.yaml`:

```yaml
als:
  factors: 128
  iterations: 15
  regularization: 0.01
  alpha: 40

train_split: 0.70
val_split: 0.15
test_split: 0.15
```

# G Artifacts

| File | Size | Description |
|---|---|---|
| `als_item_factors.npy` | 1.1 GB | $(2,262,292 \times 128)$ item embeddings |
| `uri_to_id.json` | 105 MB | URI $\leftrightarrow$ ID mapping |
| `playlists.npy` | $\approx$ 200 MB | 1M playlists as integer sequences |